



LAB GUIDE

Build with Gemini · Track 3

Prototype, equip, evaluate, and deploy your own agent-first application on Google Cloud.

Introduction

In this lab, you'll learn how to build an entire production-ready agentic application using Google Cloud tools.

You will:

- Prototype an agent using [agents-cli](#)
- Equip your agent with an enterprise-ready harness that includes Memory, Sessions, Tools, Persistent Storage, and code execution
- Evaluate and improve the quality of your agent's responses
- Deploy your agent to Agent Platform
- Equip your agent with the power to use Google's Generative Models
- Build an agent-first frontend, hosted on Cloud Run and powered by [A2UI](#)

Set Up Your Environment

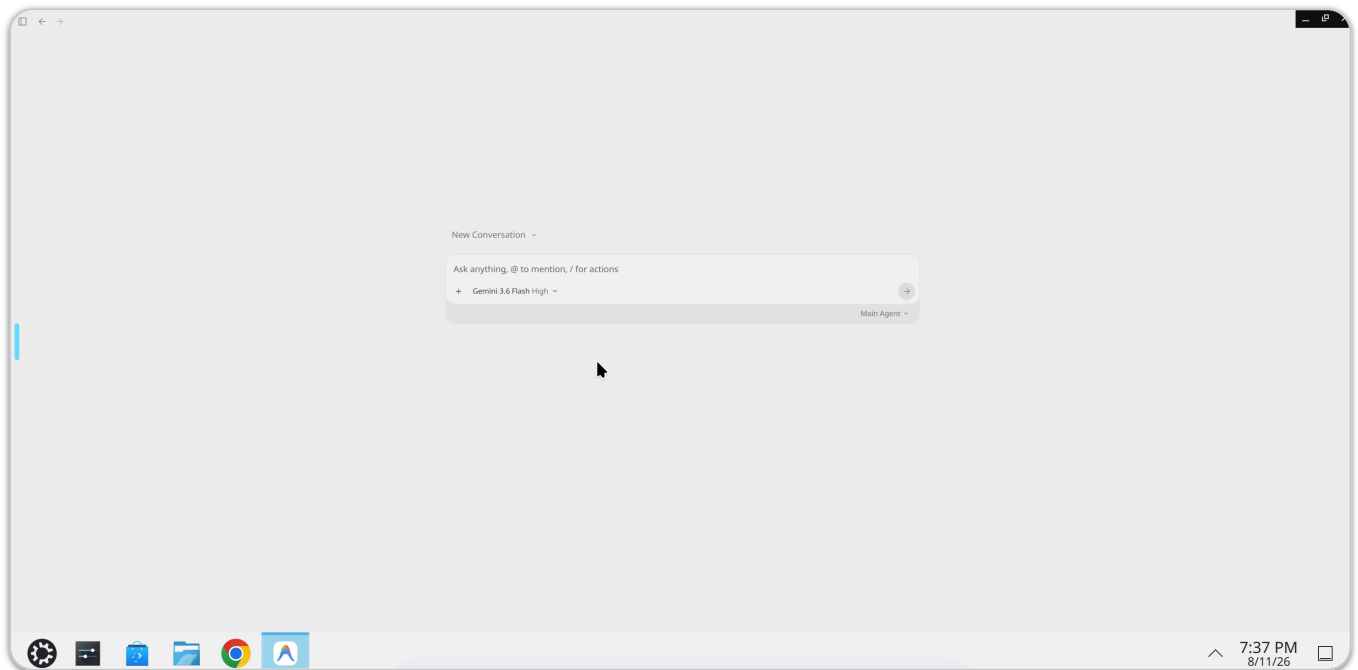
For this lab, you'll be given access to a temporary GCP environment plus a **remote virtual desktop** where you'll do all your development. Everything runs in the cloud — your code, your tools, and **Antigravity 2.0** (AGY for short), Google's agentic development environment.

- ❶ **What is Antigravity?** Antigravity 2.0 is a standalone Google app where you work with AI agents that can run commands, read and write files, search the web, and use tools. In this lab, you'll use it to build, test, and deploy your agent.

- ❗ **Avoid agent confusion!** In this workshop, we'll be *building* an agent with the help of an *additional* agent — specifically, the coding agent Antigravity (AGY). It's helpful to keep this distinction in mind as we go.

Open Antigravity 2.0

Antigravity 2.0 is already open and waiting. **Make sure you start with a new conversation before continuing:**



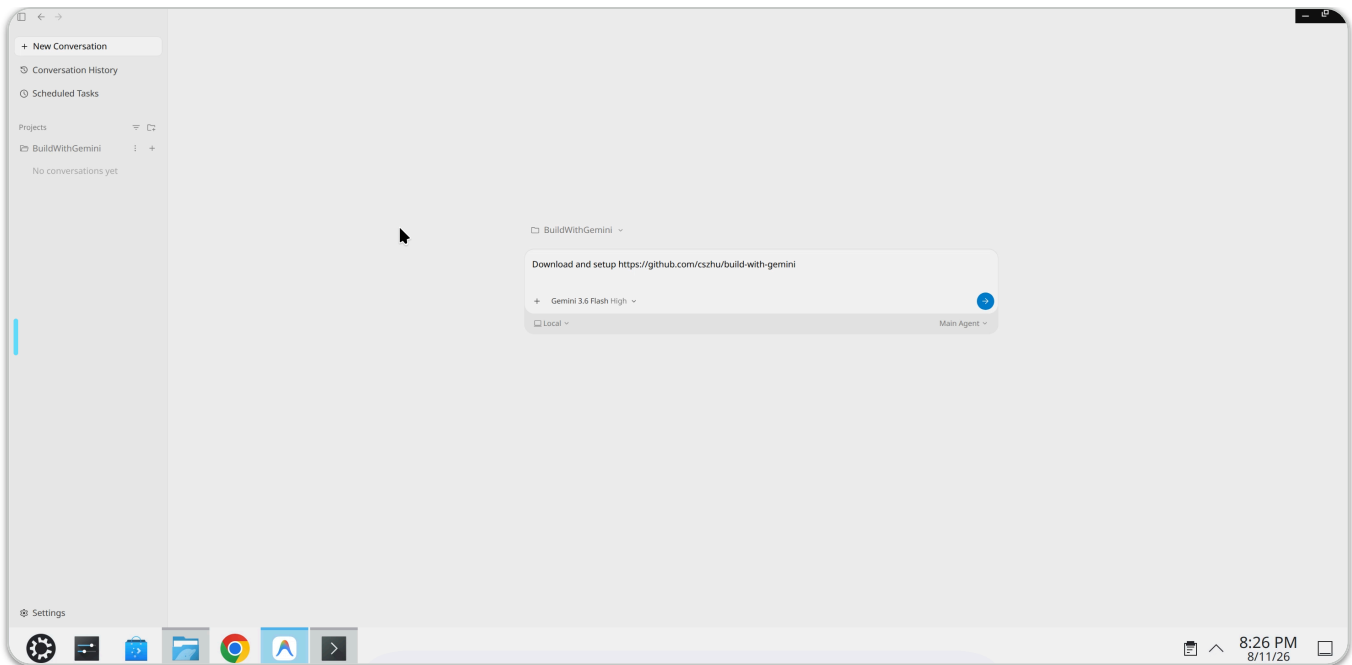
Install the lab's skills and MCPs

Before we start building, we'll set up a starter repo from GitHub that contains everything this lab needs: pre-written **skills**, **MCP** server configuration, and setup instructions for Antigravity.

In the Antigravity chat, tell it to download and set up the repo:

Download and setup <https://github.com/cszhu/build-with-gemini>

COPY

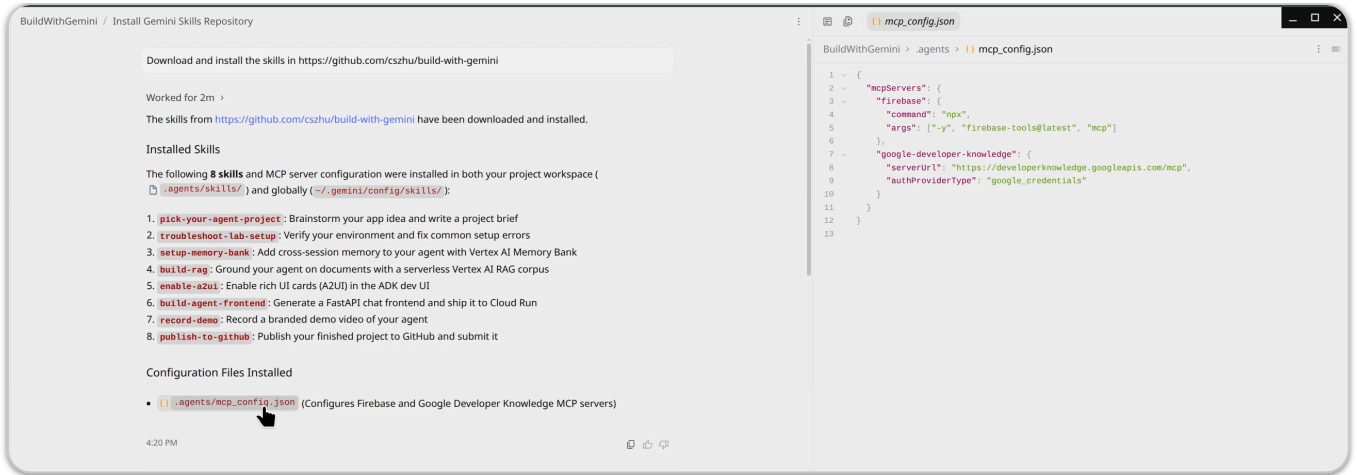


Antigravity may ask permission before it runs `git clone`. Allow it — choosing an "always allow" option saves you from re-approving `git` commands later:



What did I just install? The repo ships an `.agents/` folder that configures Antigravity for this lab. It contains **skills** (bundles of instructions that teach the agent to do a specific task well — they load automatically when relevant, so you get expert behavior without spelling out every step), **MCP** server config (Model Context Protocol — the standard way to plug external tools and data sources, like Google's official docs or a database, into the agent), and setup instructions that tell Antigravity how to install it all and configure your credentials automatically.

When it's done, Antigravity will report the skills and MCP configuration it installed. You can click any file it mentions — like `.agents/mcp_config.json` — to inspect it:



You should end up with several skills specific to this lab, plus two MCP servers:

- **Developer Knowledge MCP** — gives AGY **grounded, current knowledge of Google's official docs** (Google Cloud, Firebase, ADK, and the Agent Platform products you'll use later) instead of guessing commands, which keeps the later modules out of trial-and-error spirals.
- **Firebase MCP** — lets AGY work directly with Firebase services like Firestore, which you'll use for persistent storage.

Your setup is complete — let's start building! 🧑🏫

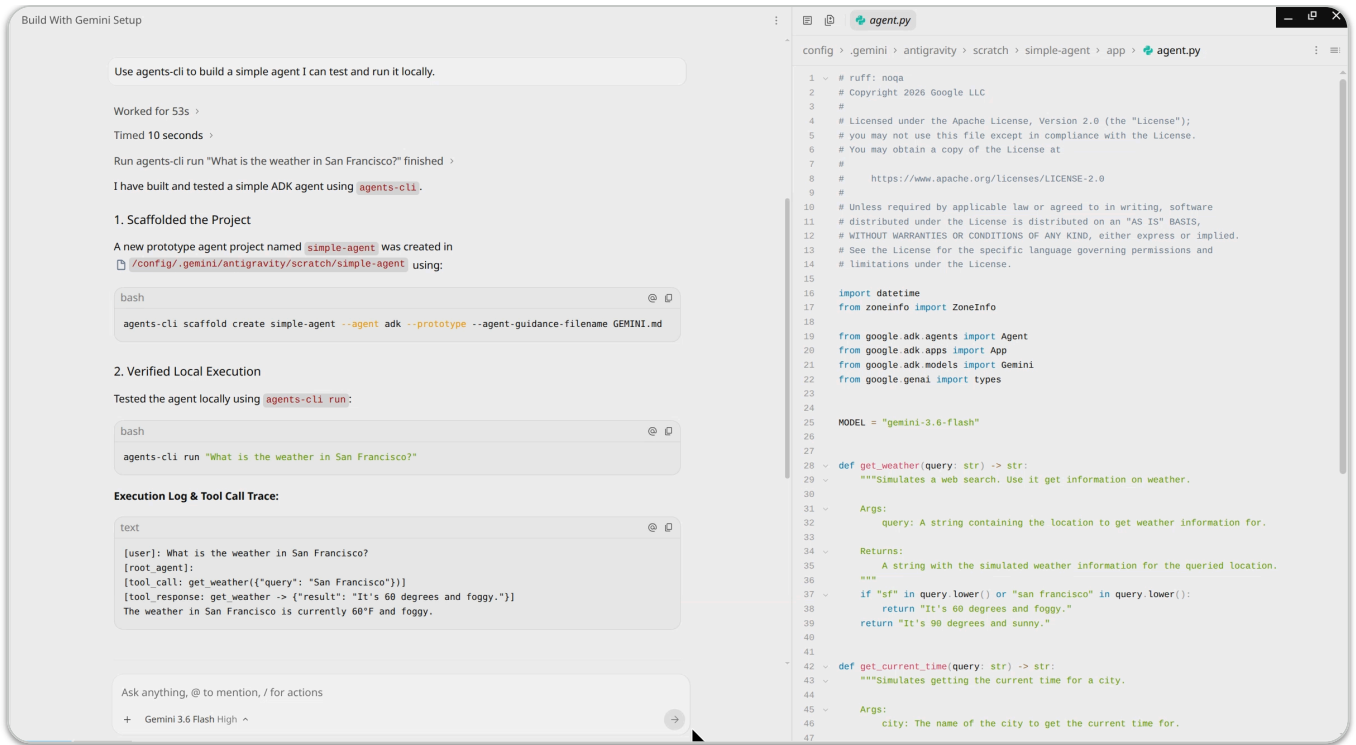
Build Your First Agent

Later in this lab, you'll design your own bespoke agentic application. But for now, let's build and test a basic agent together. Tell AGY to build a basic agent and run it locally.

Use `agents-cli` to build a simple agent I can test and run it locally.

COPY

When it's done, AGY should produce something like:



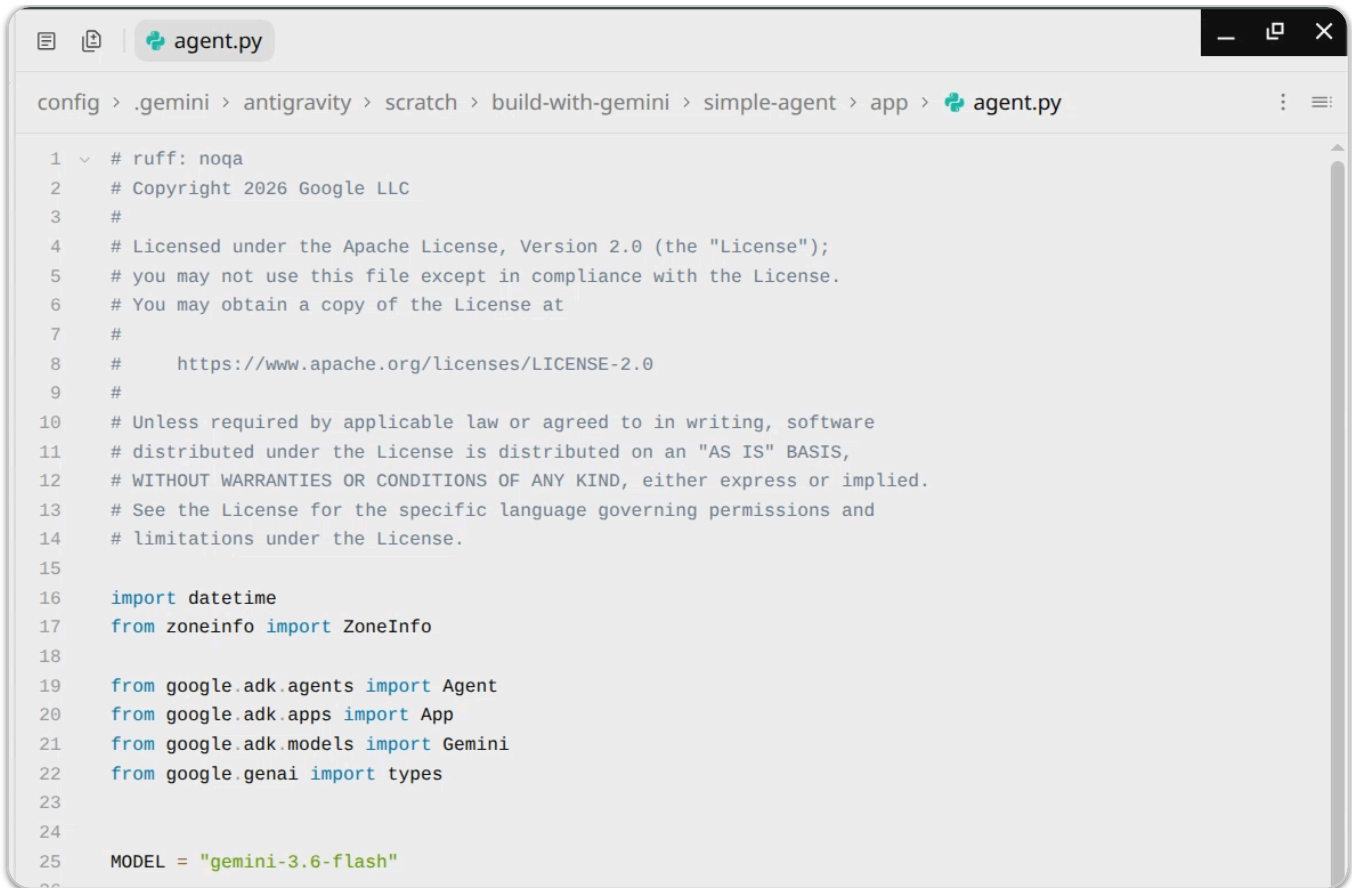
Behind the scenes, `agents-cli` scaffolds a complete ADK project for you: a Python agent definition, its configuration, and the local run/playground setup — all the boilerplate you'd otherwise write by hand — so you can go straight to shaping your agent's behavior.

We can then tell AGY to test our agent from the command line:

Test my agent with the message, "What's the weather in New York?"

COPY

We can find the logic for our agent in the file `app/agent.py` :



```

1  # ruff: noqa
2  # Copyright 2026 Google LLC
3  #
4  # Licensed under the Apache License, Version 2.0 (the "License");
5  # you may not use this file except in compliance with the License.
6  # You may obtain a copy of the License at
7  #
8  #     https://www.apache.org/licenses/LICENSE-2.0
9  #
10 # Unless required by applicable law or agreed to in writing, software
11 # distributed under the License is distributed on an "AS IS" BASIS,
12 # WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 # See the License for the specific language governing permissions and
14 # limitations under the License.
15
16 import datetime
17 from zoneinfo import ZoneInfo
18
19 from google.adk.agents import Agent
20 from google.adk.apps import App
21 from google.adk.models import Gemini
22 from google.genai import types
23
24
25 MODEL = "gemini-3.6-flash"

```

 **Tip:** If you navigate away from your terminal and AGY exits, you can always access past sessions by opening AGY again and running `/resume` .

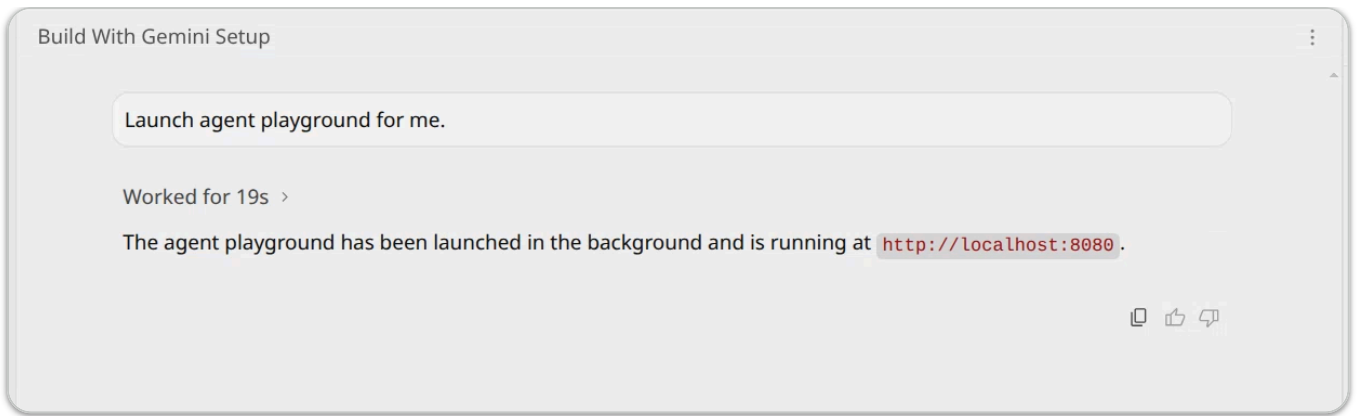
The Playground

We can also test our agent in the **Playground** — a local, browser-based chat interface. Beyond just chatting, it gives us visibility into how our agent actually works: we can inspect **sessions**, step through the **traces** of each turn, and debug the tool calls and responses behind every answer. Tell AGY to:

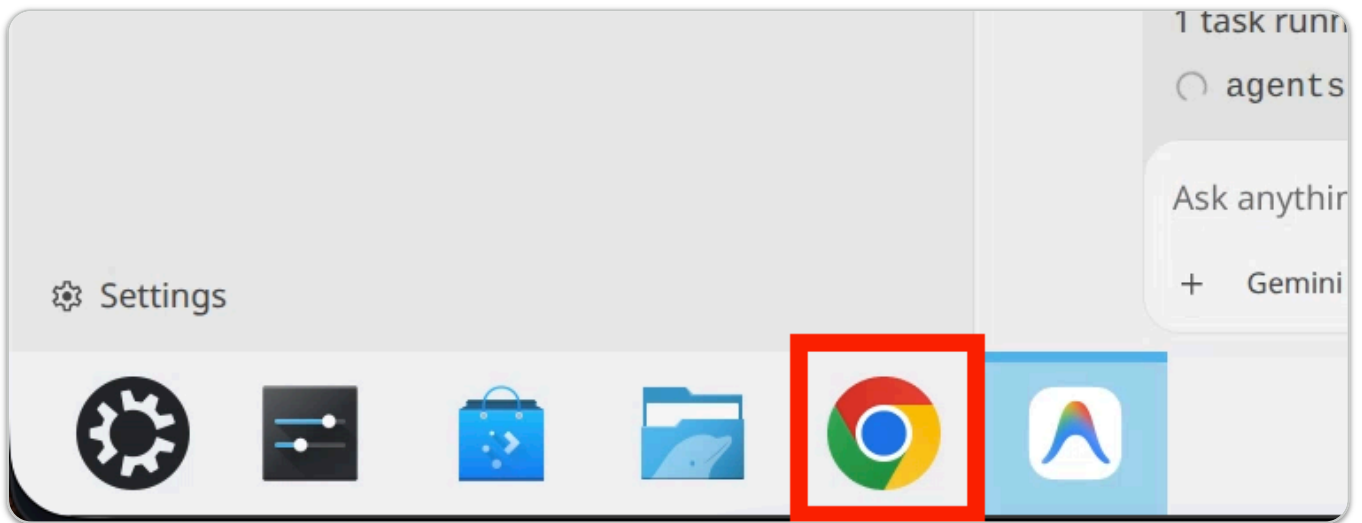
Launch agent playground for me.

COPY

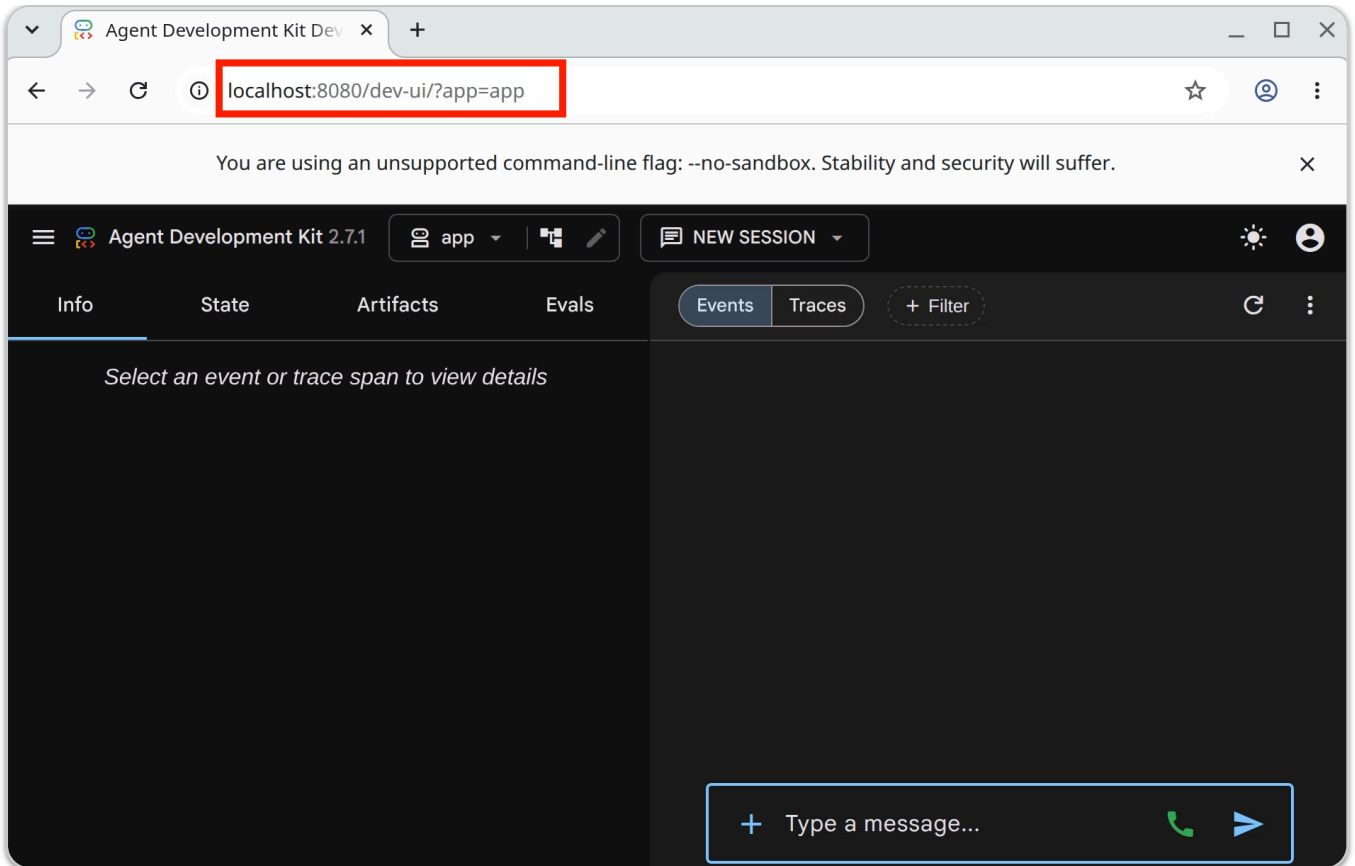
AGY launches the Playground in the background and reports the local URL it's running on — usually <http://localhost:8080> :



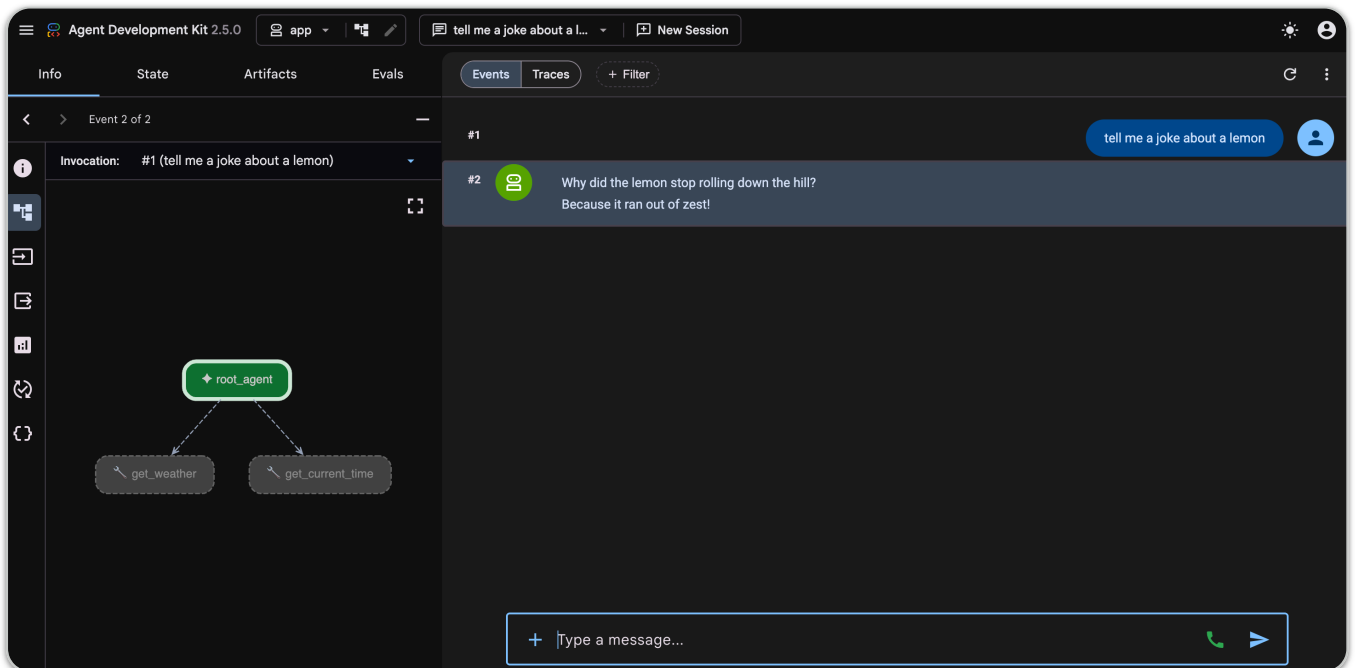
To open it, use the browser inside your remote desktop. Click the **Chrome** icon in the desktop's taskbar:



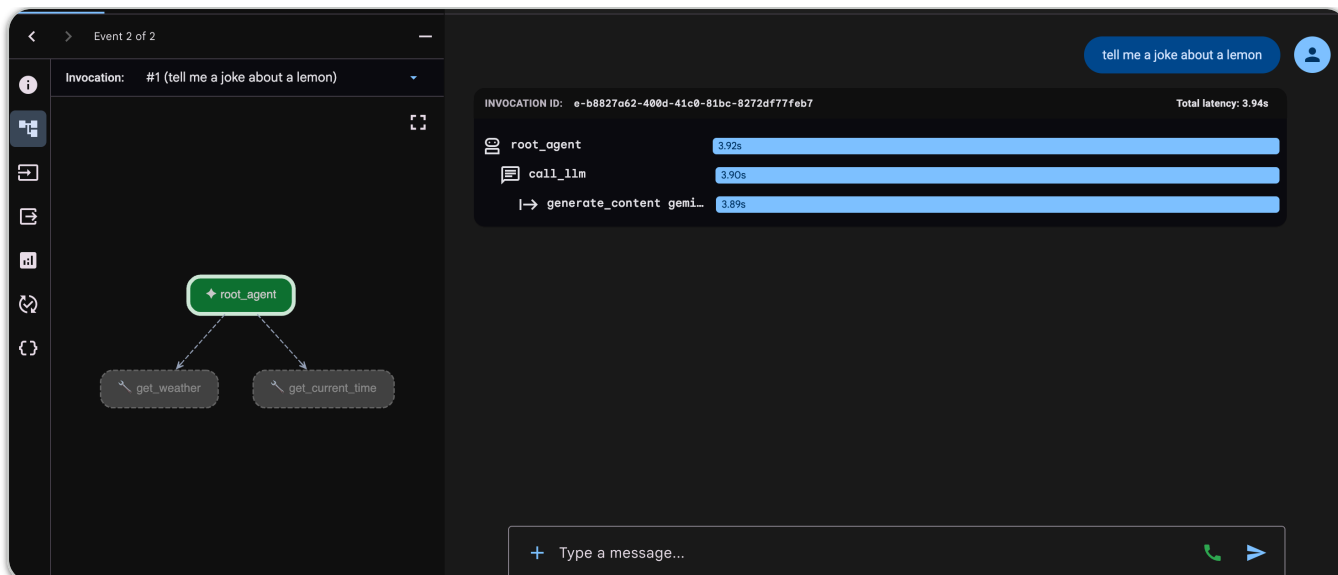
Then paste the Playground URL (<http://localhost:8080>) into Chrome's address bar and press Enter. You'll land on the ADK Dev UI, where you can chat with your agent:



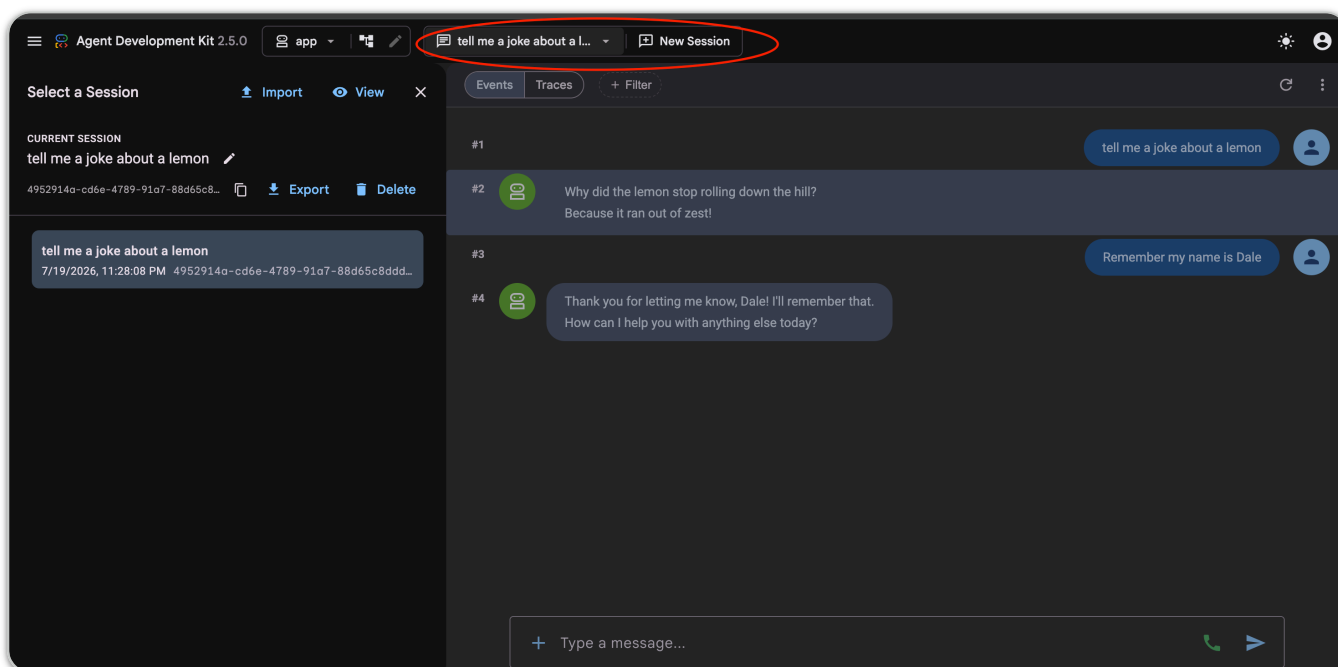
Here you can chat with our agent from a UI. On the left hand side of the screen, we can see what tools our agent has access to. Depending on how you told AGY to create an agent, your agent may have some basic tools (in this case, `get_weather`). These are blocks of deterministic, functional code your agent can call to complete tasks. We'll add more later.



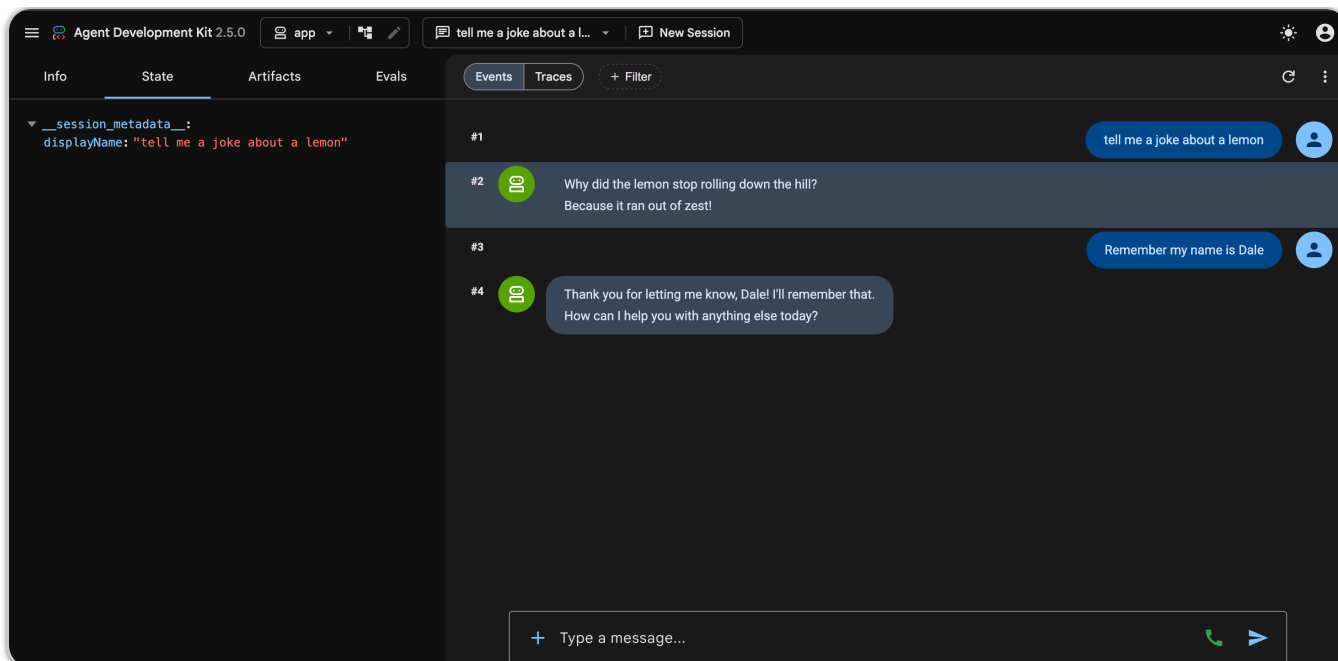
In the `Traces` tab, we can see the exact sequence of actions our agent took before producing a reply:



On the top bar of the screen, we can view our existing sessions and create new ones.



Sessions consist of the log of messages in a current conversation, as well as a scratchpad of temporary state which can view in the state tab:



Deploy to Agent Platform

Deploying our agent to Google Cloud is as simple as telling Antigravity to deploy our agent to Agent Platform:

Deploy my agent to agent platform.

COPY

Deploy my agent to agent platform.

Worked for 24s >

Deployment to Vertex AI Agent Runtime has been initiated!

Deployment Summary

- **Project:** [redacted]
- **Region:** `us-central1`
- **Target:** Agent Runtime (`agent_runtime`)
- **Task ID:** `task-89`

Agent Runtime container builds and deployments typically take 3–6 minutes to complete. I will monitor the background task and update you as soon as the deployment finishes!

Run `agents-cli deploy` for `simple-agent` finished >

Worked for 27s >

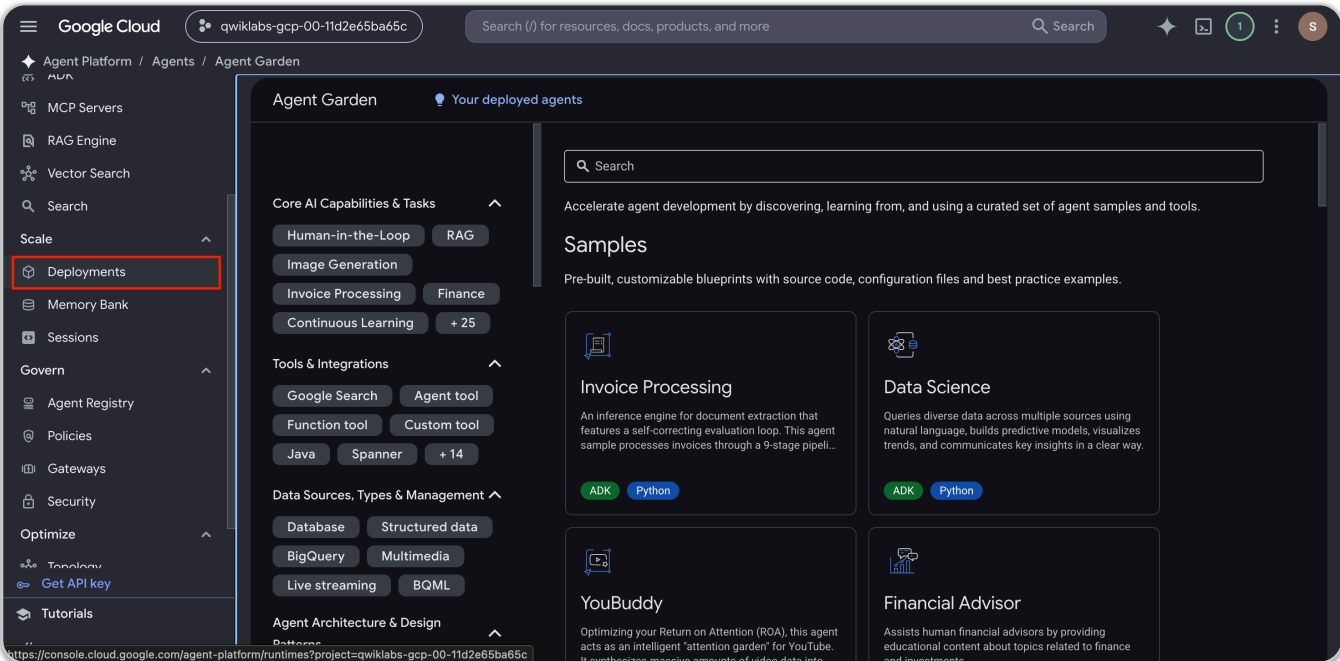
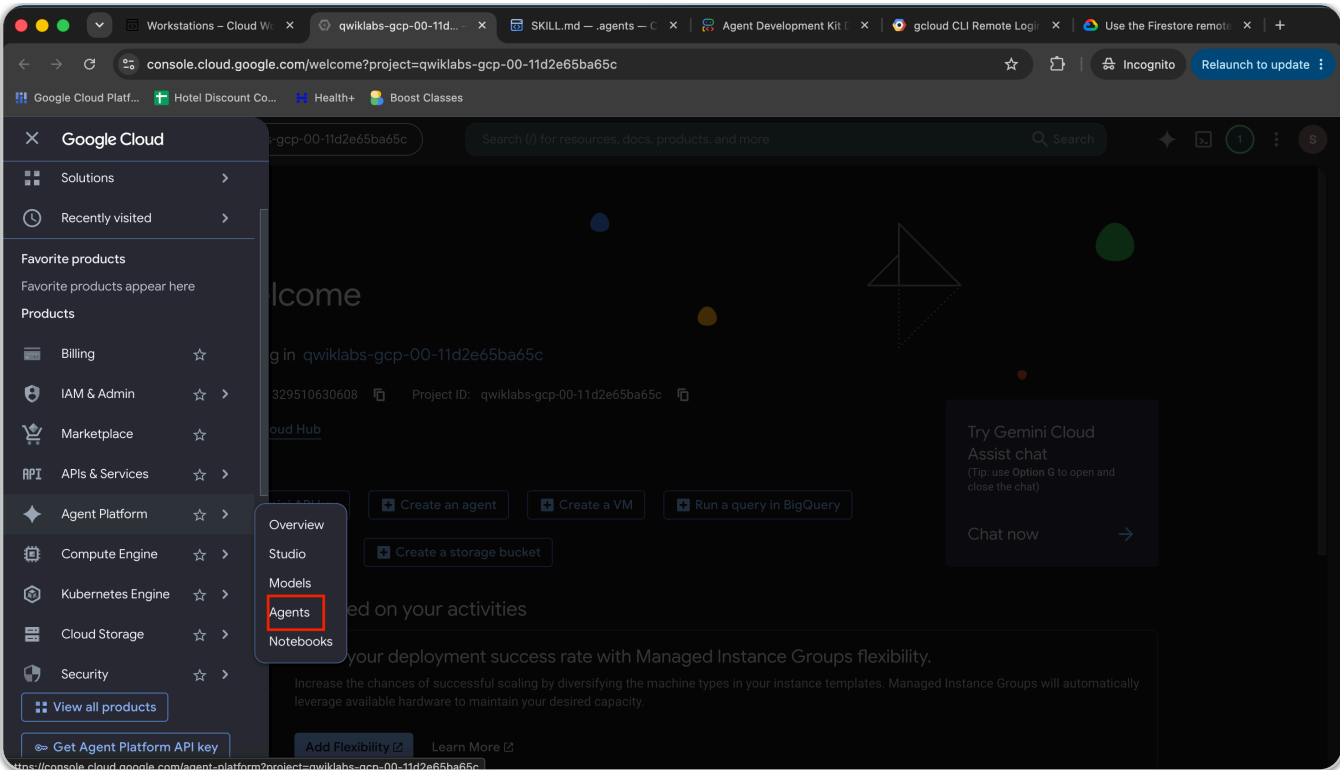
Test query on deployed Agent Runtime instance finished >

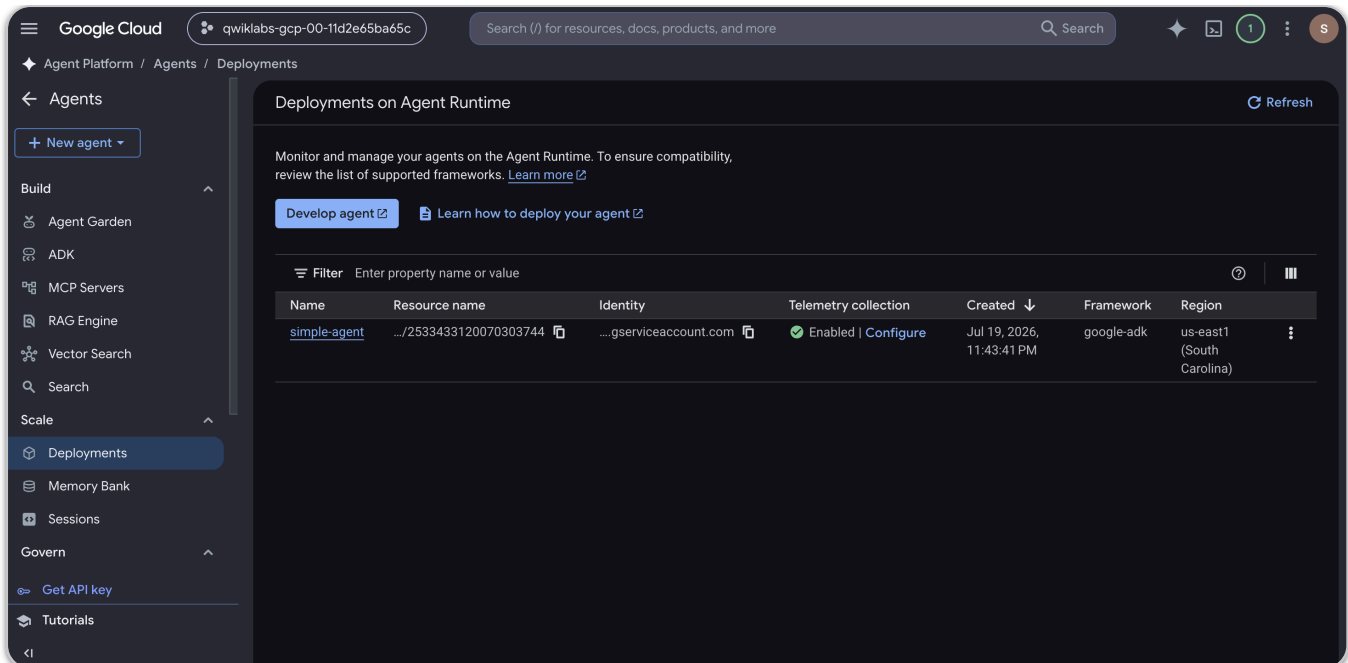
 **Your agent has been successfully deployed to Vertex AI Agent Runtime!**

Deployment Details

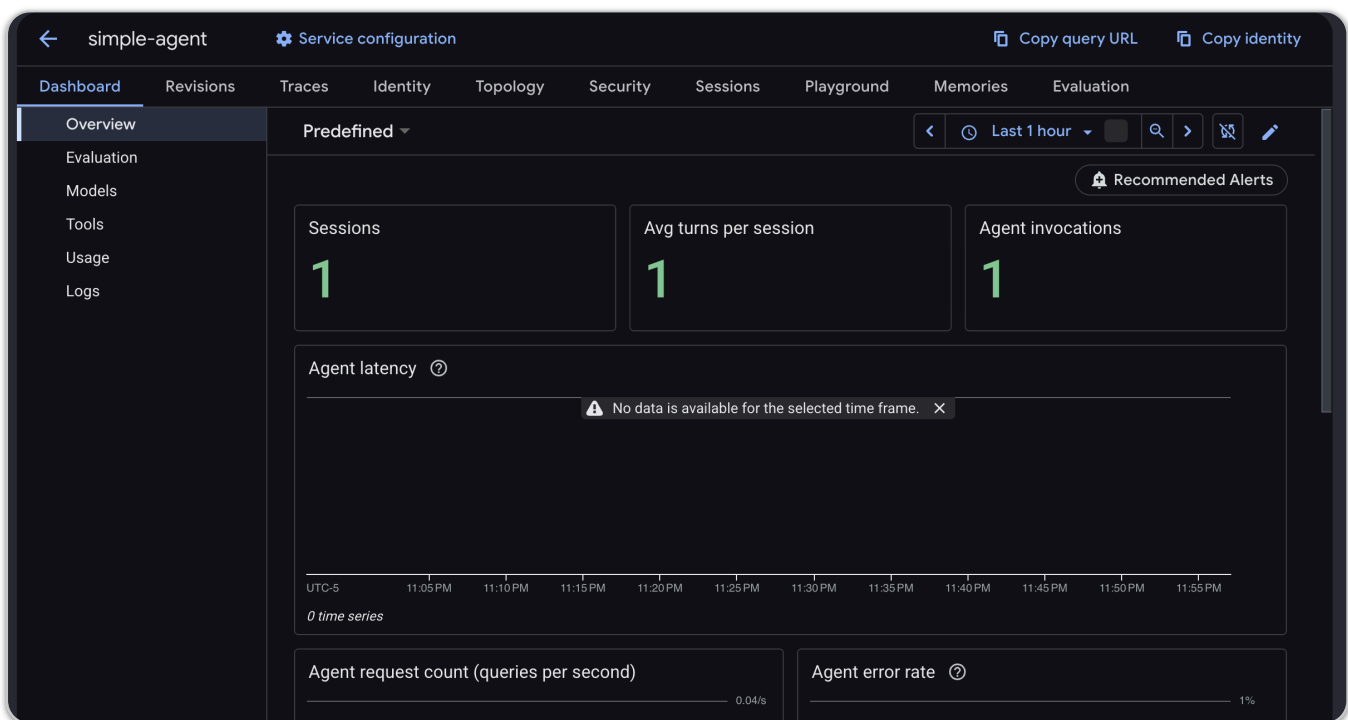
- **Project ID:** [redacted]
- **Region:** `us-central1`
- **Deployment Target:** `agent_runtime` (Managed Vertex AI Agent Engine)
- **Agent Resource Name:** `projects/[redacted]
central1/reasoningEngines/4008266890278338560`
- **Service Account:** `[redacted]@gserviceaccount.com`
- **Agent Card URL (A2A):** [agent-card.json](#)
- **Google Cloud Console:** [View Agent Engine in Console](#)

This could take a minute. When your agent is finally deployed, you'll be able to view it in Agent Platform:





Clicking in, we can see the extended tool suite available for monitoring our agent in production:



From here on, test locally. Until the **Redeploy your finished agent** step, you'll build and test everything locally in the Playground, so you don't need to redeploy after each change. Paste this into AGY once so it doesn't redeploy on its own:

For the rest of this lab, don't redeploy my agent to Agent Platform or my frontend to Cloud Run unless I explicitly ask. After each change, I'll test locally.

COPY

Design Your App

So far we've built and deployed an extremely simple agent--basically, a chatbot. In this next section, we'll evolve **that same agent** into an application of your own design, augmenting it with capabilities like:

- Calling tools (i.e. running code, calling an API, interacting with a database)
- Conversation Memory (for storing information about a user between sessions)
- Database/blob storage access for external application data (i.e. users, product inventory, images)
- The ability to generate images
- A snazzy frontend

Now's a good spot to pause and decide what kind of agentic application you'd like to build. It doesn't have to be complicated — a good workshop project is one narrow task with a bit of data behind it. For example:

- A **recipe assistant** that suggests meals from ingredients you have on hand and saves your favorites.
- A **book concierge** that recommends titles, tracks your reading list, and generates cover art.
- A **workout coach** that builds a plan, logs your sessions, and charts your progress.
- A **travel planner** that assembles an itinerary and illustrates each destination.
- A **plant-care helper** that tracks your plants, reminds you when to water, and diagnoses problems from a photo.

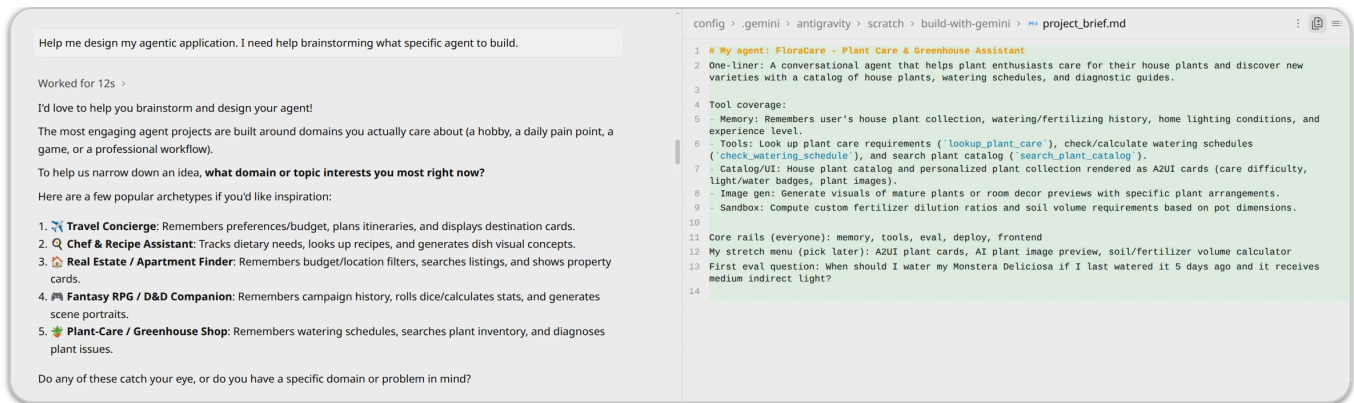
Notice these all share the same shape you'll build here: a conversation, some stored data, a tool or two, and generated media. Whatever you pick, it'll use those same ingredients.

Once you have a basic idea (or if you need help coming up with one), ask AGY for help designing your agentic application. This will invoke the [pick-your-agent-project](#) skill. In AGY, describe whatever you'd like to build, or swap in your own idea:

Help me design my agentic application. I need help brainstorming what specific agent to build.

COPY

When you're done, AGY will output a [project_brief.md](#) file for your review. Edit it yourself or with AGY until you have something you like:



Our agent will use this for reference as we continue development.

Rename your agent

Now that you have a brief, rename your existing agent project to match the app you're building:

Use the newly created `project_brief.md` to rename my existing agent project to match it: rename the project folder and update the name in `agents-cli-manifest.yaml` and `pyproject.toml`. Keep the code in `app/` unchanged, and don't deploy or change any agent logic yet.

Add Persistent Storage

Your app also needs somewhere to keep its actual data — the things it looks up and writes to, like a product catalog, a booking, an inventory count, or generated images. For that, you'll want dedicated persistent storage.

Depending on your application, you may require an external data store, like:

- **Structured data** (inventory, orders, user records) → a database like **Firestore** (serverless, simple).
- **Raw files/blobs** (bulk images, static assets) → **Cloud Storage (GCS)**.

Let's start by implementing structured data in Firestore. Antigravity is already equipped with the [Firebase MCP](#) (pre-configured back in **Set Up Your Environment**), so it can work with the Firestore API directly.

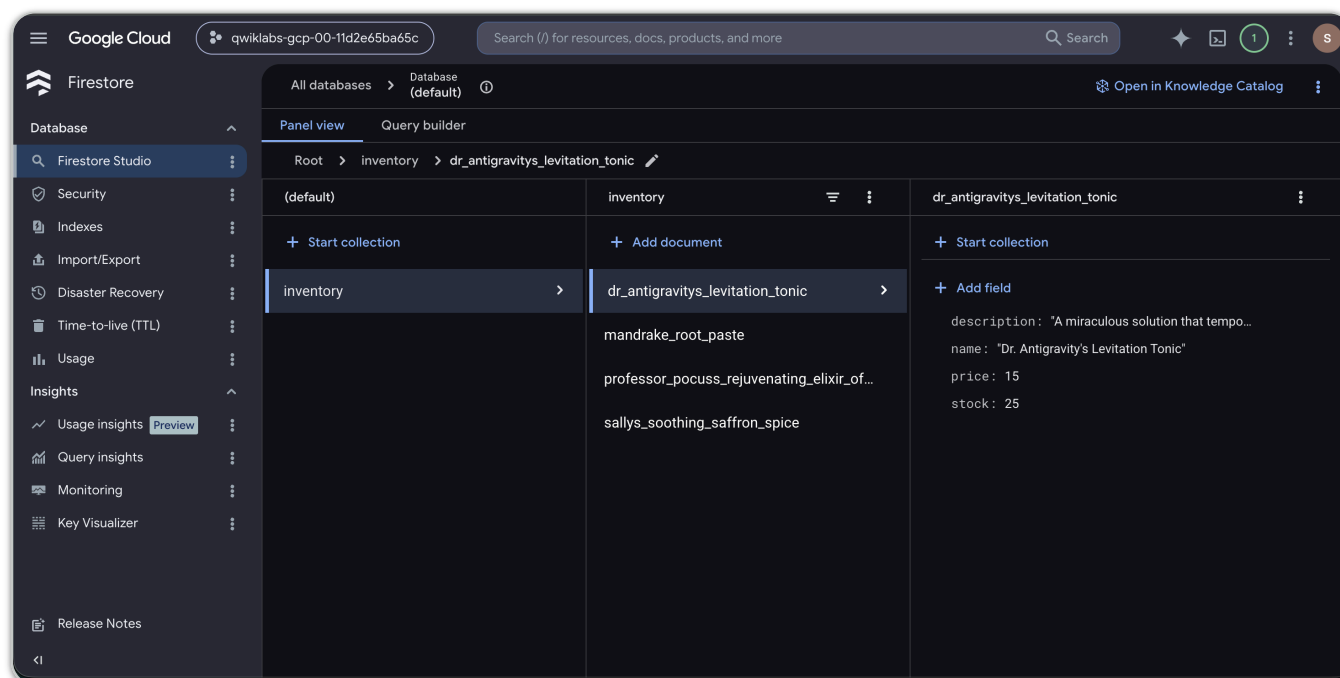
Ask AGY to give your agent a Firestore backend. Describe a collection that fits whatever you're building. AGY can look at your `project_brief.md` and pick sensible fields. Adapt this prompt to your own domain:

Give my agent a Firestore backend: a collection that fits my app (look at my ^{COPY} project_brief.md) with a few sensible fields, function tools to read and write it, and a few seeded items. Important: hardcode my project ID as a string for the Firestore client and the seed script (find it with `gcloud config get-value project`). Don't read it from `google.auth.default()` or `GOOGLE_CLOUD_PROJECT`; on Agent Platform those return the project number, which breaks Firestore after you deploy.

For example, a store/marketplace agent might use an "inventory" collection with name, description, price, and stock; a travel agent might use a "destinations" collection; a recipe agent a "recipes" collection.

In the Playground, we can verify this worked by asking the agent something it can only answer by querying the database — adapt it to your domain (for a store: *"What items do you have in stock?"*; for a travel agent: *"What destinations do you know about?"*).

You can watch the data live in your Firestore database [here](#). A nice property of Firestore is that it's **real-time**: as your agent writes (say, an action that updates a record), the document updates instantly in the browser. What's more, you can edit entries in the browser and see them reflected immediately in the agent.



Store files in Cloud Storage

Firestore is great for structured records, but for raw files — images, audio, video — you'll want **Cloud Storage**. This is handy when your agent generates media that you want to persist and serve publicly, for example so your frontend can embed it as an image.

Ask AGY to create a bucket. Adapt the name to your app:

```
Create a Cloud Storage bucket for this project. Give it a name that fits my
app (look at my project_brief.md; add a short random suffix if the name is
taken), and set the permissions so objects can be viewed publicly (i.e.
embedded in a web page as images).
```

COPY

To confirm your bucket was created, find it here: <https://console.cloud.google.com/storage>.

In the image-generation step below, your agent uploads each generated image to this bucket and gets back a public URL. A public URL is a plain <https> link that any browser can load, so the image can be displayed anywhere your agent's responses show up — including the rich UI cards and custom frontend you'll build later in this lab.

Add Tools

Right now, your agent can only talk. Tools are functions that your agent can use to actually do things and execute. Some tool examples are: looking up data from a database, taking an action (like sending an email), creating something. This is when your agent stops just being a chatbot and starts actually doing things.

Every agent is different, so start by asking your coding AI assistant for tools that fit the agent you've decided to build:

```
Look at my project_brief.md and the agent I'm building. Suggest 2-3 tools it
could call to take real action or fetch real data, then recommend the
simplest one to implement first.
```

COPY

After discussing and planning with your agent, pick one to implement.

```
Implement the tool we just discussed as a function tool and add it to my
agent. Keep the implementation minimal.
```

COPY

Test it in the agent playground or debug screen. Ask it to do something that the tool is able to do, and the agent should be able to call the tool. You should be able to confirm this trace in the logs.

Call External APIs

The tool you just added ran your own code. But a tool can also reach out to the wider web and pull in live data from someone else's service. That's the difference between an agent that *mocks* data (invents a plausible-looking answer like "here's a sample weather report") and one that returns the real thing. If your app needs facts you don't have (the weather, a plant's care instructions, a location on a map, an exchange rate), there's probably an API for it, and wiring it into a tool is often a single HTTP call.

For example, if you're building a gardening app, [Perenual](#) has 10,000+ plant species (watering, sunlight, hardiness zones) behind a free API key.

Two kinds of API you'll run into:

- **Free, no-key APIs.** Call them straight from a tool with a plain HTTP request, no signup. Great for prototyping.
- **Key-required APIs.** You register (usually free for low volume), get an API key, and send it with each request.

Where to find free APIs

You don't have to know an API exists already. A couple of directories are worth browsing:

- [public-apis](#) (~390k GitHub stars). ~50 categories in one big table, with an [Auth](#) column ([No](#) means no key needed).
- [APIsList](#). Hand-checked, with filters for free, no-key, and open-source APIs, plus a live/broken status on each.

Some need no signup at all. For example, `curl "https://api.open-meteo.com/v1/forecast?latitude=40.71&longitude=-74.00¤t_weather=true"` returns live weather, and `curl "https://api.frankfurter.dev/v1/latest?base=USD"` returns exchange rates.

Let AGY do the searching and wiring. It can look at your project, pick an API that fits, and build the tool that calls it:

Look at my `project_brief.md`. Find a free public API that fits my app's domain (browse the `public-apis` directory at <https://github.com/public-apis/public-apis>), then add a function tool that calls it and returns real data. Keep the

COPY

```
implementation minimal, and if the API needs a key, read it from an
environment
variable rather than hardcoding it.
```

Test it in the Playground by asking something that can only be answered from the live API. In the trace you'll see the tool fire, the HTTP call go out, and the agent answer from real data instead of making it up.

Example: Google Maps

Location features like "find coffee shops near me," "how far is it," and "what's the address of this place" are a natural fit for **Google Maps Platform**, and a good example of a key-required API. Agents like to *mock* this ("the nearest cafe is 0.3 miles away"), so calling the real API is a genuine upgrade.

Maps needs an API key, and the project needs **billing enabled** first. Without billing, requests are throttled to about one a day; with it, each API has a monthly free tier before it starts charging. The APIs you're most likely to want are **Geocoding** for turning an address into coordinates, **Routes** for distances and travel times, and **Places (New)** for searching real places.

To get a key, in the [Cloud Console](#):

1. Enable the APIs you need, for example [Geocoding API](#) and [Places API \(New\)](#). (Or let AGY enable them for you; see the prompt below.)
2. On the [Credentials](#) page, click **Create credentials → API key**.
3. Restrict the key to those APIs. For a backend, use an IP address restriction rather than HTTP referrers.
4. Copy the key. You'll hand it to AGY in the next step so it can wire it in as the `GOOGLE_MAPS_API_KEY` environment variable. Keep it out of your source files; the `publish-to-github` skill flags a hardcoded key before it reaches GitHub.

Then have AGY enable the APIs, store your key, and build the tool. Paste your key in place of `PASTE_KEY_HERE` :

```
First, enable the Geocoding and Places (New) APIs on my project. Here's my
Maps
API key: PASTE_KEY_HERE. Save it as GOOGLE_MAPS_API_KEY in my local .env;
don't
hardcode it in a source file. Then add a tool that uses the
Geocoding API to turn an address into coordinates, and one that uses the
```

[COPY](#)

Places

API (New) to find nearby places of a given type. Call the REST endpoints directly and return the key fields (name, address, location). Use the Developer Knowledge MCP to confirm the exact endpoints and required headers.

Test it with something location-based ("Find parks near Seattle") and watch the tool hit the live Maps endpoint in the trace.

- ❗ **In the lab environment:** enabling Maps needs billing on the project. If your temporary lab project can't, use a free no-key API from above for now and come back to Maps on your own Google Cloud project later (new accounts get a \$300 free trial credit that covers it). The pattern is the same for any key-based API: enable it, get a key, store it in an env var, call it from a tool.

Generate Images

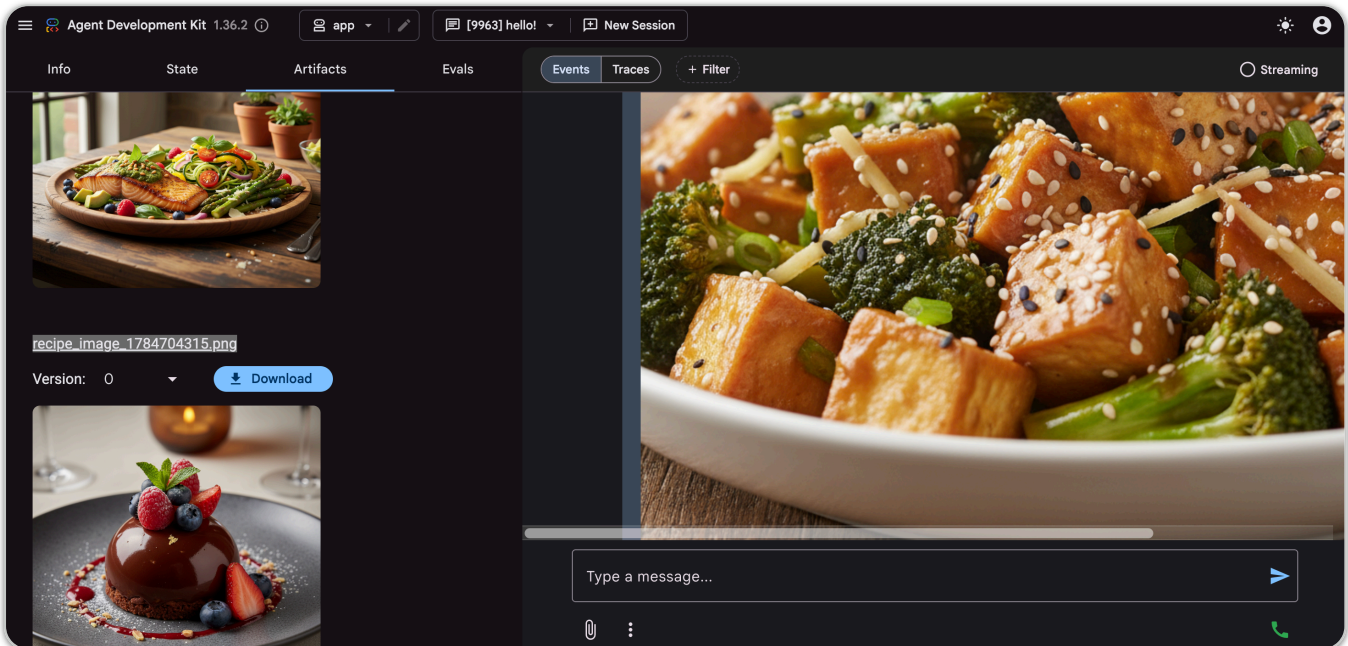
Tools can also give your agent the power of Google's generative AI models. A great one to reach for is image generation with [gemini-3.1-flash-lite-image](#) (Nano Banana 2 Lite), Google's fastest image model, which turns a short text prompt into an image so your agent can create visuals on demand. It runs in the [global](#) region. You can read about it here: <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/3-1-flash-lite-image>

Add a tool that generates an image for an item in my agent's domain (look at my project_brief.md) using the [gemini-3.1-flash-lite-image](#) model in the global region. Do two things with the generated image: (1) save it with `tool_context.save_artifact` so it shows up in the Playground's Artifacts panel, and (2) upload the same image bytes to the public Cloud Storage bucket I created earlier and return its public https URL (`https://storage.googleapis.com/<bucket>/<object>`) from the tool. Hardcode the bucket name as a string, the same way we hardcoded the Firestore project. Do not write the image to a local file and return a path. Use the Developer Knowledge MCP to confirm the API if you're unsure.

The tool does two things because each output has a different use. The saved artifact shows in the Playground's Artifacts panel, which is a quick way to see the image while you develop. The public URL is what you use later: your agent can drop it into the rich UI cards you'll add

later in this lab to show the picture, and your frontend can load it. The Artifacts panel only works inside the Playground, so the public URL is what makes the image visible elsewhere.

Now ask your agent to generate an image for one of the items in your domain. You should get back a real image, which appears in the Artifacts panel.



Run Code in a Sandbox

Some agents need to execute code, whether that's for quick API calls, data analysis, or just doing math. Agent Platform offers code execution in an isolated sandbox, so your agent is able to code safely. Since the state persists across sessions and it doesn't touch your host, it is a very safe way to run model-generated code, even in enterprise settings.

Prereqs: the Agent Platform API enabled. When you run locally, the sandbox uses your own credentials.

```
Add Agent Platform code execution to my agent using  
AgentEngineSandboxCodeExecutor so it can safely run Python in a sandbox. If  
I don't already have a sandbox, create one from the Agent Engine in my  
deployment_metadata.json.
```

[COPY](#)

Test it by asking a question that requires real computation or analysis, like fizzbuzz, and confirm the agent runs code in the sandbox and answers from the result.

Of course, you are able to update your tool instructions from the last section to use the sandbox as well. For folks who are building agents for things like auto-trading or coding, this

is a super useful feature to incorporate into your agent.

Add Memory

While **sessions** store information about a current conversation, sometimes we want to remember facts *between* sessions. Example: if a customer tells a shopping agent he hates the color red, that fact should be present across *all* future sessions.

We can enable this cross-session remembering with Agent Platform's **Memory Bank**. Every time the user sends the agent a message, Memory Bank analyzes the conversation and automatically extracts and remembers factual snippets that might be useful to future conversations. Unlike Firestore, which stores your app's data, Memory Bank remembers facts about the user.

Note: Memory Bank is a managed service on Agent Platform, but you don't need to redeploy to use it. The Agent Engine you created in your first deploy also serves as your Memory Bank, and your local Playground can connect to it directly. The standard `agents-cli playground` command can't connect to Memory Bank, so AGY starts the Playground with `adk web` instead. It still opens at <http://localhost:8080>.

Let's tell AGY to set up memory using its skill:

Use the `memory-bank-setup` skill to add memory to my agent. Reuse the Agent Engine from my earlier deployment as the Memory Bank (its ID is the last part of `remote_agent_runtime_id` in `deployment_metadata.json`). Also set the memory service in the app code so it's used when I redeploy later, but don't redeploy now. Then restart my local playground with ``uv run adk web . --port 8080 --reload_agents --memory_service_uri=agentengine://<ID>`` instead of ``agents-cli playground``, and use that same command whenever I ask you to launch or restart the playground.

COPY

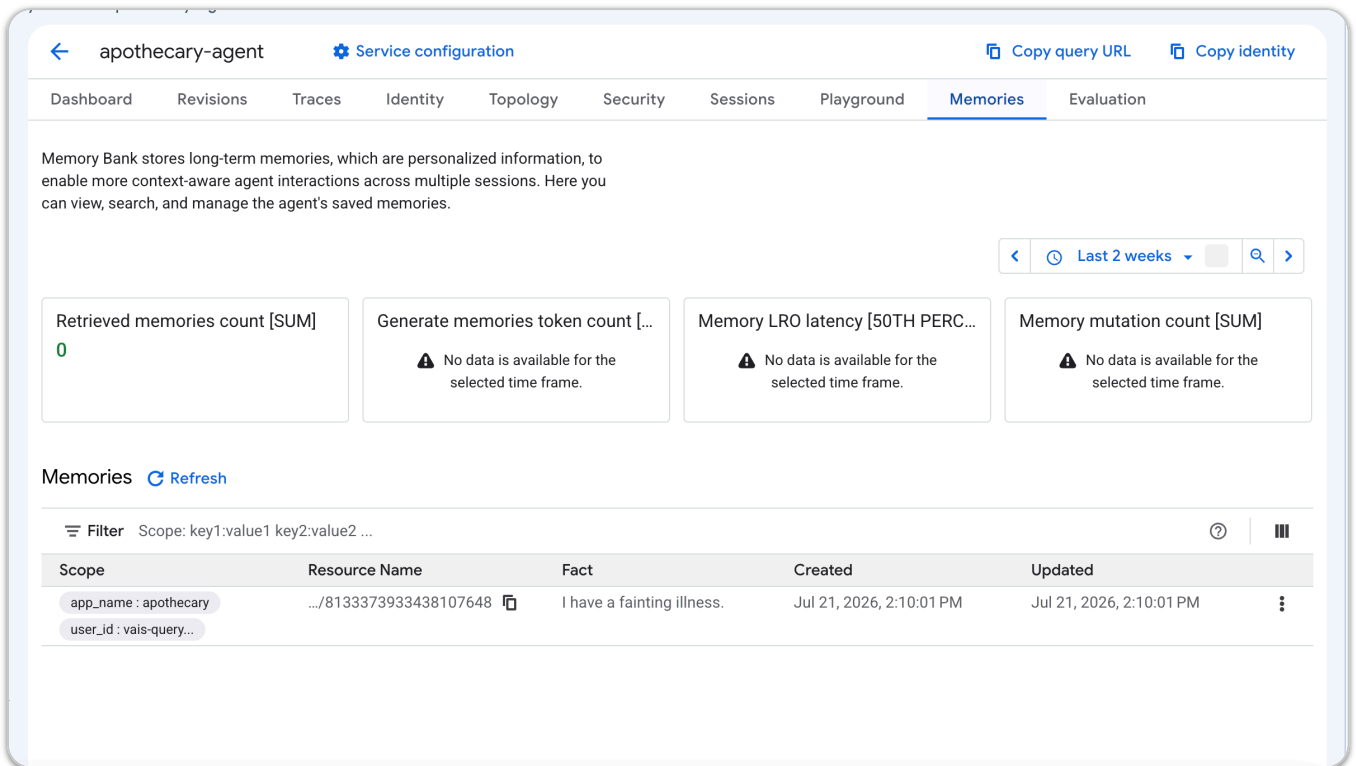
Depending on your application, you can be more specific about what to remember:

Configure memory so that all user allergies are remembered.

COPY

To try it out, tell your agent a durable fact about yourself in the Playground (for example, a preference), then start a **new session** and ask something that depends on it. Memory Bank

extracts memories in the background after each turn. You can see the stored memories in the Agent Platform UI [here](#):



If you don't see any memories, ask AGY to confirm the Playground was started with `--memory_service_uri`.

Enrich Responses with A2UI

So far your agent replies in plain text. A2UI lets it reply with real UI instead: cards and tables. Your agent writes a small description of the UI as JSON, and a renderer turns that JSON into components on screen. A2UI is open source and works with any agent built with ADK. (In the ADK dev UI the cards are **display-only** — buttons and forms show but don't do anything; keep your UI to cards and tables built from rows and columns.)

There are two pieces to set up, and you need both, or you will just see raw JSON:

1. A system prompt that teaches the model to output valid A2UI JSON. The `a2ui-agent-sdk` builds this for you with `A2uiSchemaManager`.
2. A small callback (`a2ui_utils.py`) that wraps the model's JSON in the format the ADK dev UI's built-in renderer expects. You add it to your agent as an `after_model_callback`.

Ask AGY to set up both. This uses the `enable-a2ui` skill, which ships the callback file for you:

Use the `enable-a2ui` skill to add A2UI to my agent: build the system prompt with `A2uiSchemaManager` (version 0.8) and the Basic Catalog, copy in `a2ui_utils.py`, and wire it up as an `after_model_callback`. COPY

One thing to watch: the callback only recognizes v0.8 messages, so if the schema manager emits v0.9 instead, the callback never fires and you're back to raw JSON.

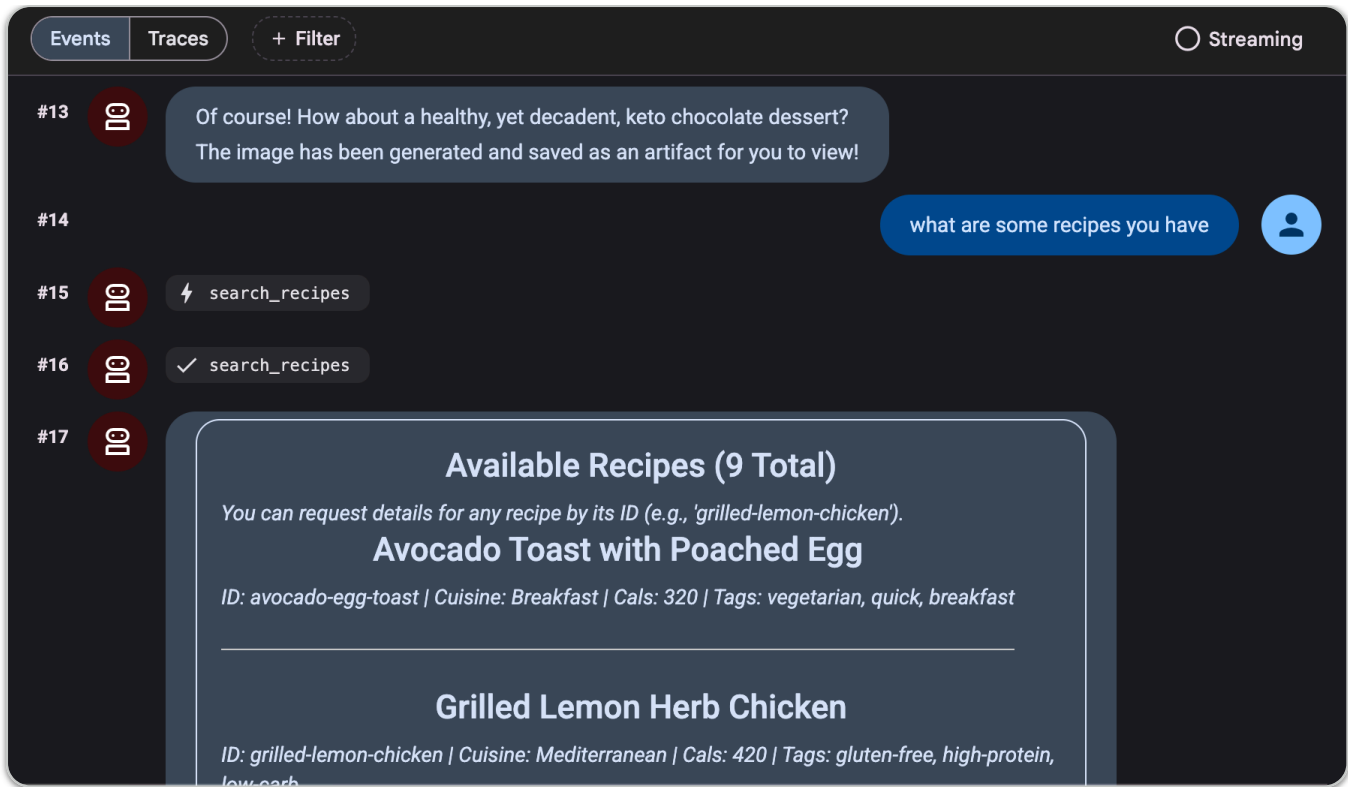
Validate A2UI locally before redeploying

Test it in the same Playground you have been using. The Playground is really the ADK dev UI (`agents-cli playground` runs `adk web` for you), and it comes with a built-in A2UI renderer. It only renders A2UI once the callback is in place, so restart the Playground to pick up your changes:

Restart the agent playground so it picks up my latest changes. COPY

This restart is **local only**: it reloads the Playground process on your machine, not the agent you already deployed to Agent Platform. Redeploying takes several minutes, so don't do it after every small change — verify A2UI here first. You'll push everything you've built (storage, tools, media, sandbox, memory, and A2UI) to the deployed agent in one shot later, in the **Redeploy your finished agent** step.

Start a new session, then ask for something that fits a card or table. For a store agent, "Show me what's in stock" might render a table, and "Tell me about this item" might render a card. Adapt the question to whatever your agent knows about. You should see real UI instead of JSON.



A card can also show an image. Add an [Image](#) component and set its URL to the public URL your image tool returns, and the picture appears inside the card instead of only in the Artifacts panel. Ask your agent to generate an image for an item and show it, and you should get a card with the picture in it. The URL must be a public <https> link. A bare artifact filename cannot be loaded by the renderer and shows as a broken image.

One important setting: **turn Token Streaming OFF** in the Playground (the toggle in its settings). With streaming on, the dev UI shows the raw streamed JSON and never swaps in the card. If you still see raw JSON after that, check that the callback is wired up and that you are on version 0.8, then hard-refresh and start a new session. (If the Playground in your environment still won't render it, you can run the dev UI directly with `uv run adk web --port 8080 --allow_origins "*" --reload_agents`, plus the `--memory_service_uri` flag from the Add Memory step.)

Build a Frontend

Up to now you've tested everything in the ADK playground, and that's been your agent's "face" while you built out storage, tools, media, and A2UI. Now, let's give your agent a shareable web face and ship it to the cloud.

Redeploy your finished agent

Your agent has grown a lot since you first deployed it: it now has storage, tools, media generation, a sandbox, memory, and A2UI, none of which are in that original deployment. Before wiring up a frontend, push the latest version to Agent Platform so the frontend talks to the fully-featured agent:

```
Redeploy my agent to Agent Platform. If my agent reads any API keys from  
environment variables (like GOOGLE_MAPS_API_KEY), pass them to the  
deployment with --update-env-vars.
```

COPY

Do this next if your agent uses Firestore or Cloud Storage. The deployed agent runs as its own service account, which has no access to your data by default. It worked in the Playground because you ran as yourself. On the deployed agent a Firestore query comes back empty, and an image upload to Cloud Storage fails and no image appears. Grant the roles the deployed agent needs:

```
Grant my deployed agent's service account the roles it needs:  
roles/datastore.user for Firestore, and roles/storage.objectAdmin on my  
image bucket if it generates images.
```

COPY

This uses the [troubleshoot-lab-setup](#) skill to grant those roles to your agent's runtime service account.

Grab the resource name from the fresh [deployment_metadata.json](#) . You'll point the frontend at it in a moment.

Build the frontend

We don't hand you a pre-built frontend. Instead, ask AGY to build a minimal one for you. This invokes the [build-agent-frontend](#) skill, which copies a small FastAPI proxy and chat UI into a [frontend/](#) folder. The browser only ever talks to the proxy (same origin, no CORS), and the proxy authenticates to your deployed agent with Application Default Credentials.

The chat shows your agent's text replies and renders its A2UI cards. The same cards you saw in the ADK dev UI now show up in your own web app, including any generated images (the card's [Image](#) points at the public URL from your Cloud Storage bucket). The template ships a small built-in renderer, so it works whether or not your agent uses A2UI (anything it can't render falls back to plain text).

```
Using the build-agent-frontend skill, copy its minimal FastAPI proxy and  
chat UI template into ./frontend and wire it to my deployed agent using
```

COPY

```
AGENT_ENGINE_RESOURCE_NAME and AGENT_DIRECTORY (my agent_directory from
agents-cli-manifest.yaml). Keep it a plain chat UI. Do not build a React app
or pull in a large sample frontend.
```

(If your project already has a web frontend you'd rather keep, tell AGY to adapt that one to the same proxy pattern instead of creating a new one.)

Test locally

Before deploying the frontend, confirm that your code can reach your agent that is currently deployed on Agent Platform. Running locally means the frontend server runs on your machine instead of Cloud Run, but it talks to the same deployed agent. This catches wiring issues before you ship to Cloud Run.

Ask AGY to install the frontend and start it locally, pointed at your deployed agent:

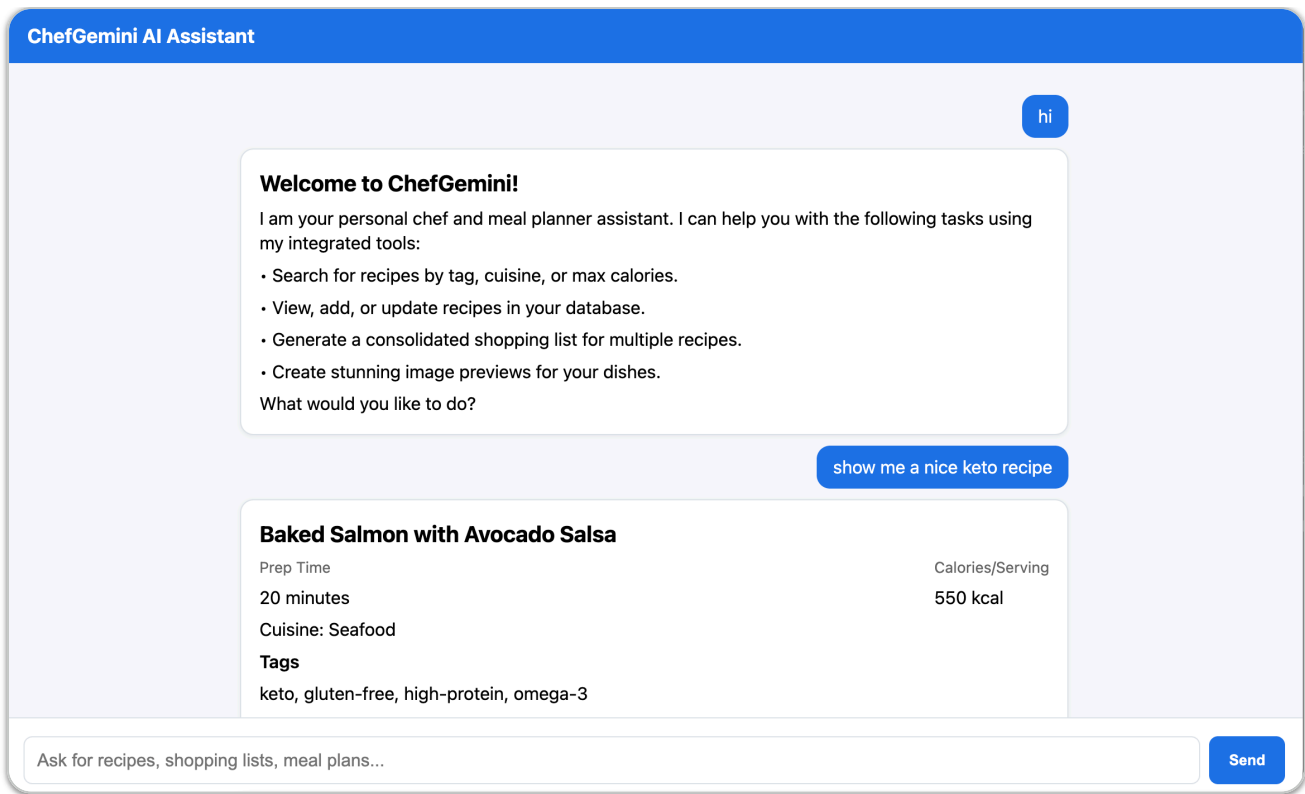
```
Run my frontend locally from the frontend/ folder: install its dependencies,
set AGENT_ENGINE_RESOURCE_NAME to the resource name in
deployment_metadata.json and AGENT_DIRECTORY to my agent_directory from
agents-cli-manifest.yaml, then start the server on http://localhost:8080.
```

If you would rather run it yourself, from the `frontend/` folder:

```
pip install -r requirements.txt
export AGENT_ENGINE_RESOURCE_NAME="<paste the resource name from
deployment_metadata.json>"
export AGENT_DIRECTORY="app"
python main.py
```

Then test it in your browser:

1. Open <http://localhost:8080> and you should see your chat UI.



2. Send a message and wait for a reply. Then, in the same chat, ask "What did I just ask?" to confirm the session is wired up correctly. If it can't remember, the frontend is not reaching your deployed agent, so recheck the resource name from the earlier step.

Deploy to Cloud Run

Finally, ship the frontend so anyone can use it.

Note that the frontend that we are shipping deploys differently from the agent. You used `agents-cli` to deploy the agent to Agent Platform, the frontend is a separate custom web app, so you need to ship that to Cloud Run with `gcloud`. Essentially, the agent lives in Agent Platform while the frontend lives in Cloud Run and talks to the agent. This is the recommended pattern for a custom UI.

The Cloud Run service runs as a different service identity than you did locally, so its service account needs `roles/aiplatform.user` or `/chat` will error out.

```
Deploy the frontend to Cloud Run pointing at my AGENT_ENGINE_RESOURCE_NAMECOPY
and AGENT_DIRECTORY, and grant the Cloud Run service account
roles/aiplatform.user so it can reach the agent.
```

Or run it directly:

```
gcloud run deploy <your-frontend-name> \  
  --source . \  
  --region <your-region> \  
  --allow-unauthenticated \  
  --set-env-  
vars="AGENT_ENGINE_RESOURCE_NAME=$AGENT_ENGINE_RESOURCE_NAME,AGENT_DIRECTORY  
=$AGENT_DIRECTORY"
```

COPY

Open the Cloud Run URL and chat with your deployed agent. That's the full loop: agent on Agent Platform, frontend on Cloud Run, talking to each other.

Customize Your Frontend

Your whole UI lives in one file, [frontend/static/index.html](#) : a chat page with a title, a header, and an accent color. Ask AGY to restyle it. Check each change at <http://localhost:8080> (refresh the page), and when you're happy with it, ask AGY once to redeploy the frontend to Cloud Run. A few things to try:

Rebrand my frontend: set the title and header to my app's name and change the accent color.

COPY

Add a row of 3 clickable example prompts above the input, tailored to my agent (look at my `project_brief.md`).

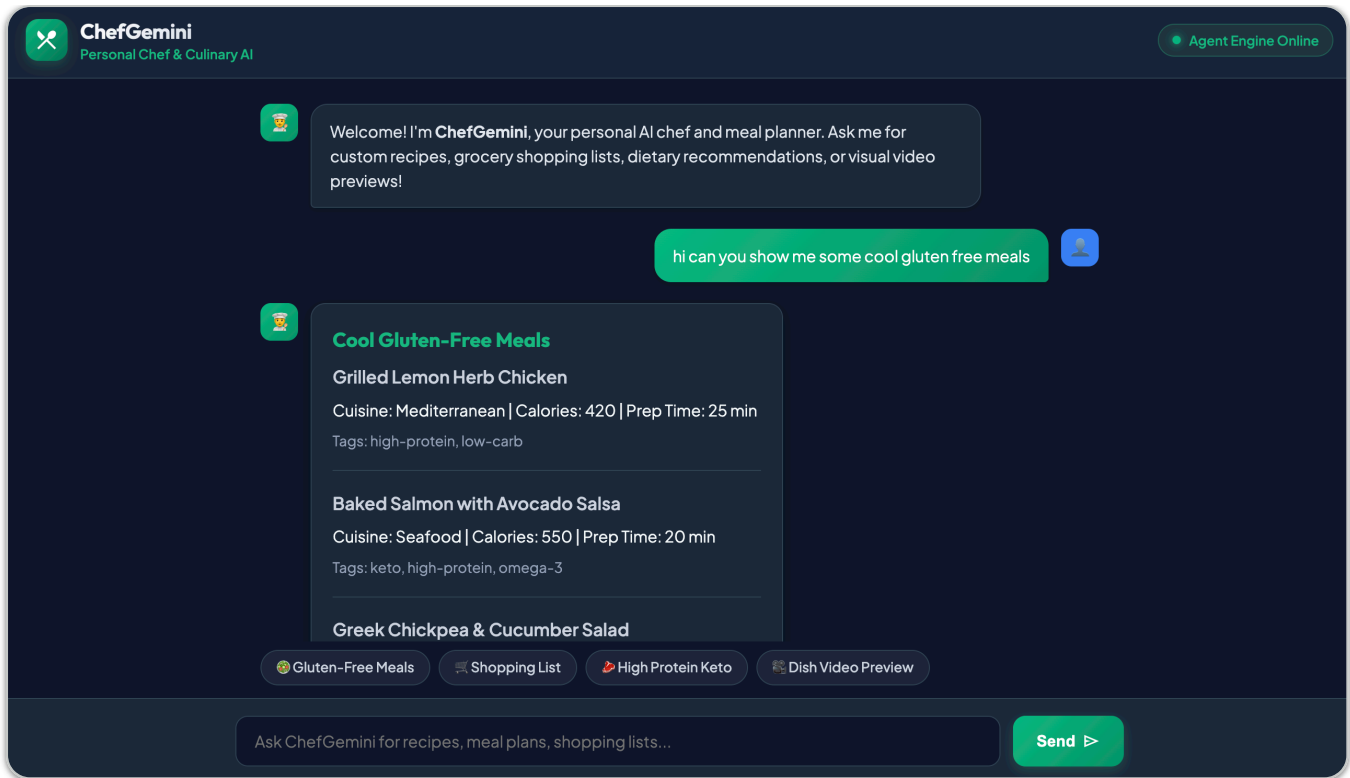
COPY

Create a nice dialogue layout for my app that matches my theme.

COPY

Suggest some UI improvements we can make to the frontend.

COPY



Stretch Goals

You've built the full loop. To take it further, pick from the stretch menu in your `project_brief.md`. A few ideas:

- Video previews with Omni: generate a short video clip instead of a still image.
- Check out Cloud Trace: your deployed agent already exports traces, so you can see the full execution flow — every LLM call and tool call — for each request.
- Keep customizing the frontend to match your brand.

Omni ([gemini-omni-flash-preview](#)) is Google's video model: it generates video from text or a reference image, and you can keep editing the result with follow-up prompts instead of starting over from scratch. It also reasons about physics and real-world context, so motion in the clip looks physically plausible instead of uncanny.

To try video generation with Omni:

COPY

```
Add a tool that generates a short video for an item in my agent's domain
(look at my project_brief.md) using Google's Omni model (gemini-omni-flash-
preview) in the global region. Do two things with the generated video: (1)
save it with tool_context.save_artifact so it shows up in the Playground's
Artifacts panel, and (2) upload the same video bytes to the public Cloud
Storage bucket I created earlier and return its public https URL
(https://storage.googleapis.com/<bucket>/<object>) from the tool. Hardcode
```

```
the bucket name as a string, the same way we hardcoded the Firestore project. Do not write the video to a local file and return a path. Use the Developer Knowledge MCP to confirm the API if you're unsure.
```

Share What You Built

You've gone from an empty folder to a deployed, agent-first application. The last step is the fun one: capture it and show it off. Your repo ships a `record-demo` skill that films your agent in action and produces a polished, branded video clip — perfect for a tweet, a Slack message, or the top of a README.

Record a demo clip

The `record-demo` skill drives your chat UI in a headless browser: it opens the page, types real prompts, waits for each reply, and saves a short `.webm` with a "Gemini World Tour" frame around it. It works against either face of your agent — your custom frontend (`localhost:8080`) or the Playground.

First, make sure the UI you want to film is running (your frontend from the last section, or the Playground). Then tell AGY what to show off — pick 2–3 prompts that exercise your agent's best features (a tool call, a Firestore lookup, a generated image, an A2UI card):

```
Record a demo video of my agent. Show it doing the thing my app does best,  
then ask a second, richer prompt that shows off a tool call, a database  
lookup, or a generated image. COPY
```

For example, a store agent might use: *"Record a demo of my agent. Ask it what's in stock, then ask it to generate a picture of the first item."* A great clip is short and snappy — **2–3 turns, roughly 20–40 seconds**.

- ① **Add a soundtrack (optional).** The recorder can score your clip with AI-generated instrumental music using Google's Lyria model, matched to the video's length. Just describe the vibe:

```
Record a demo of my agent and add some upbeat lo-fi background  
music. COPY
```

If you'd rather run it yourself, call the recorder directly from your project root (bump `--wait` to `30000` + for turns that generate an image or video):

```
node .agents/skills/record-demo/record-agent.js \  
  -q "First demo prompt" \  
  -q "Second demo prompt" \  
  -o agent_demo.webm
```

COPY

On success it prints `SUCCESS: Recording saved to <path>`, leaving a branded `.webm` in your project — ready to share.

Turn it into a shareable README

A demo is even better with a landing page. Ask AGY to generate a README that tells the story of your app and shows the clip you just recorded — as a **GIF**, which is the no-fuss way to get a demo playing in a README:

```
Generate a README.md for my project. Describe what my agent actually does  
based on the code in this repo – read app/ and agents-cli-manifest.yaml to  
see which tools and Google Cloud services are really wired up (Memory Bank,  
Firestore, Cloud Storage, image generation, A2UI) and list ONLY those. Don't  
claim a capability the code doesn't implement; if something in my  
project_brief.md was planned but not finished, leave it out or mark it  
clearly as "planned, not yet implemented." Do NOT put any live links to  
localhost (like http://localhost:8080) or to ephemeral Cloud URLs (Cloud Run  
/ Agent Engine endpoints from this lab) in the README – they die when this  
workstation is torn down and will 404 for anyone reading it later. If you  
need to show how to run it, write it as setup/run instructions (the commands  
to start it locally), not as a clickable link. Then convert the demo I  
recorded with record-demo (the .webm file) to an optimized, looping GIF and  
embed it near the top with a relative path so it plays inline – use only  
that real recording; if there isn't one, leave the demo out rather than  
generating or drawing a placeholder.
```

COPY

Why a GIF? Committed alongside your code and referenced with a plain relative path (`! [demo](demo.gif)`), it renders and loops automatically on GitHub — no uploads, no special links. Keep the original `.webm` as your high-quality **social asset**: X, LinkedIn, and Slack all play an uploaded video natively.

❗ Prefer a real video player in the README? GitHub only renders an inline player for videos uploaded through its web editor (`.webm` committed to the repo and

linked by path stays a download link). If you want the player rather than a GIF, drag the `.webm` into the README on github.com and it inserts a working embed.

Now your repo has a proper front door: a clear description, a feature list, and a looping demo of your agent in action — the perfect thing to link when someone asks *"what did you build?"*

Publish to GitHub

This section is optional. Do it only if you want to keep your code so you can work on it later, or share it. If you're done, feel free to skip it.

Everything you've built so far lives on this remote desktop, and the desktop is temporary. When the lab ends, it goes away, and your code with it. So the last step is to push your project to **your own personal GitHub** so you keep it, can share it, and have a shot at the **Build with Gemini** project gallery.

This uses the [publish-to-github](#) skill, which creates a repo on *your* GitHub account and pushes your project to it. Tell AGY:

```
Publish my project to GitHub. Don't generate the swag submission form.
```

[COPY](#)

What happens, and where you stay in control:

1. **You sign in to your own GitHub.** AGY runs the GitHub CLI, which prints a **one-time code** and the URL github.com/login/device. Open that on *any* device (your laptop or phone), sign in to **your personal GitHub**, and enter the code. This keeps your GitHub identity separate from the lab's Google Cloud credentials.

 **Note:** sign in with *your own* GitHub account, not the Qwiklabs lab account. The repo lands on your personal profile so it's yours to keep after the lab.

2. **AGY asks before it pushes.** Before creating the repo or pushing anything, AGY confirms the repo name and shows you what it's about to do. **Nothing is pushed to your account without your say-so.** Don't blindly approve; check the repo name and that it's your account first.

- ① **Name your repo** `buildwithgemini-[YOUR-APP-NAME]` (for example, `buildwithgemini-recipe-assistant`). AGY proposes this name by default, but if you rename it, keep this format.

3. **Your repo goes public.** The repo is created **public** so it can be featured in the gallery and shared on social. AGY writes a `.gitignore` and scans for secrets before committing, so credentials don't get pushed.

When it's done, AGY prints your new repo's URL.

Earn a Skill Badge

Now that you've finished the workshop, you can earn an official **Build with Gemini** Skill Badge for your Google Developer Profile. It's an Intermediate quiz that takes about **30 minutes** and confirms what you learned here.

Claim it today:

[Claim your Build with Gemini Skill Badge](#)

Or scan the QR code to open it on your phone:



Congratulations! You've designed, built, equipped, evaluated, deployed, shared, *and* published a complete agentic application.

